Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого Институт прикладной математики и информатики

Отчет по лабораторной работе №7

Решение краевой задачи для ОДУ 2-го порядка.

Выполнил студент гр. 5030102/30004 Зернов.К.В
Преподаватель: Курц. В.В

Санкт-Петербург
2026

Содержание

Формулировка задачи и ее формализация 2
Формулировка задачи 2
Формализация задачи 2
Алгоритм метода и условие его применимости 2
Условие применимости 2
Алгоритм метода факторизации(метод расщепления) 2
Предварительный анализ задачи 3
Условия существования решения 3
Тестовый пример 3
Подготовка контрольных тестов 5
Модульная структура программы 6
Численный анализ 9
Графики 10
Вывод 13

Формулировка задачи и ее формализация

Формулировка задачи

Требуется численно решить краевую задачу для обыкновенного дифференциального уравнения (ОДУ) второго порядка с заданными начальными условием на отрезке $[a, b]$ с использованием метода факторизации(метода расщепления).

Уравнение: $y''(x) + 2xy'(x) + (1 + x^2)y(x) = (x^2 + 2x + 2)e^x$

$[a, b] = [0, 1], A = 3, B = e, \alpha_0 = 1, \alpha_1 = 2, \beta_0 = 1, \beta_1 = 0$

Точное решение: $y(x) = e^x$

Исследования, которые необходимо провести:

1. Исследовать зависимость погрешности от h. Сравнить с теоретической зависимостью.
2. Исследовать влияние ошибок в исходных данных (ГУ) на решение.

Формализация задачи

Пусть дано ОДУ второго порядка: $y'' = f(x, y, y')$, с граничными условиями:

$$\begin{cases} \alpha_0 y(a) + \alpha_1 y'(a) = A \\ \beta_0 y(b) + \beta_1 y'(b) = B \end{cases}$$

Требуется с помощью метода факторизации найти приближенное решение.

Алгоритм метода и условие его применимости

Условие применимости

Рассмотрим линейное ОДУ 2-го порядка с переменными коэффициентами

$p(x)y'' + q(x)y' + r(x)y = f(x)$

$p(x), q(x), r(x), f(x) \in C([a, b])$

Алгоритм метода факторизации(метод расщепления)

Дано: $p(x)y'' + q(x)y' + r(x)y = f(x)$ и ГУ:

$$\begin{cases} \alpha_0 y(a) + \alpha_1 y'(a) = A \\ \beta_0 y(b) + \beta_1 y'(b) = B \end{cases}$$

Предполагаем, что существуют функции $s(x)$ и $t(x)$, что решение ОДУ 2го порядка может быть представлено в виде ОДУ 1го порядка

$y'(x) = s(x)y(x) + t(x)$

Если сконструируем функцию $y(x) = u(x) + cv(x)$ для которой

$$\begin{cases} L(u)=f \\ L(v)=0 \end{cases}$$

то эта функция будет решением ОДУ 2го порядка для $\forall c$.

Функции s, t и ищутся из систем:

$$\begin{cases} s'=-s^2-\frac{q}{p}s-\frac{r}{p} \\ s(a)=-\frac{\alpha_0}{\alpha_1} \end{cases} \qquad \begin{cases} t'=\frac{f}{p}-st-\frac{qt}{p} \\ t(a)=\frac{A}{\alpha_1} \end{cases}$$

Решаем ОДУ модифицированным методом Эйлера, получаем сеточные функции s^h, t^h

Решаем задачу Коши и получаем сеточную функцию y^h

$$\begin{cases} y'(x)=s(x)y(x)+t(x) \\ y(b)=\frac{B-\beta_1 t(b)}{\beta_0+\beta_1 s(b)} \end{cases}$$

Предварительный анализ задачи

Условия существования решения

Функции $p(x), q(x), r(x), f(x) \in C([a,b])$.

Существует $s(x) \in C([a,b])$ решение уравнения Риккати:

$$s' + ps^2 + qs + r = 0$$

Для рассматриваемой задачи уравнение Риккати имеет решение: $s(x) = -x$, которое непрерывно на отрезке $[0,1]$ и не имеет полюсов. Следовательно факторизация существует, а метод применим для решения данной краевой задачи.

Тестовый пример

Уравнение: $y'' - y = 0$, $x \in [0,1]$, $h = 0.5$

$p(x) = 1, q(x) = 0, r(x) = -1, f(x) = 0$

ГУ: $\begin{cases} y(0)+y'(0)=1 \\ y(1)=e \end{cases}$

Точное значение: $y = \frac{1}{2}e^x + \frac{e^2}{2}e^{-x}$

Шаг 1: Ищем функции сеточные функции s^h и t^h, решая две задачи Коши модифицированным методом Эйлера.

a) **ищем функцию s^h:**

$$\begin{cases} s'=s^2-\frac{q}{p}s-\frac{r}{p} \\ s(a)=-\frac{\alpha_0}{\alpha_1} \end{cases} \Leftrightarrow \begin{cases} s'=s^2-\frac{q}{p}s-\frac{r}{p} \\ s(0)=-\frac{1}{1}=-1 \end{cases}$$

шаг $0 \to 0.5$:

$$\begin{cases} s_{0.5}=s_0+\frac{h}{2}f(x_0,s_0) \\ s_1=s_0+hf(x_{0.5},s_{0.5}) \end{cases} \Leftrightarrow \begin{cases} s_{0.5}=-1+0.25\cdot((-1)^2-\frac{0}{1}\cdot(-1)^2-(-\frac{1}{1})) \\ s_1=-1+0.5\cdot f(x_{0.5},s_{0.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} s_{0.5}=-0.5 \\ s_1=-1+0.5\cdot 1.25=-0.375 \end{cases}$$

Шаг $0.5 \to 1$:

$$\begin{cases} s_{1.5}=s_1+\frac{h}{2}f(x_1,s_1) \\ s_2=s_1+hf(x_{1.5},s_{1.5}) \end{cases} \Leftrightarrow \begin{cases} s_{1.5}=-0.375+0.25\cdot((-0.375)^2+1) \\ s_2=-0.375+0.5\cdot f(x_{1.5},s_{1.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} s_{1.5}=-0.08984375 \\ s_2=0.1290359497 \end{cases}$$

Получили значения $s^h = \{-1, -0.375, 0.1290359497\}$

b) **ищем функцию t^h:**

$$\begin{cases} t'=\frac{f}{p}-st-\frac{qt}{p} \\ t(a)=\frac{A}{\alpha_1} \end{cases} \Leftrightarrow \begin{cases} t'=\frac{f}{p}-st-\frac{qt}{p} \\ t(0)=\frac{1}{1}=1 \end{cases}$$

Шаг $0 \to 0.5$:

$$\begin{cases} t_{0.5}=t_0+\frac{h}{2}f(x_0,t_0,s_0) \\ t_1=t_0+hf(x_{0.5},t_{0.5},s_{0.5}) \end{cases} \Leftrightarrow \begin{cases} t_{0.5}=1+0.25\cdot 1 \\ t_1=1+0.5\cdot f(x_{0.5},t_{0.5},s_{0.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} t_{0.5}=1.25 \\ t_1=1+0.5\cdot(0.5\cdot 1.25) \end{cases} \Leftrightarrow \begin{cases} t_{0.5}=1.25 \\ t_1=1.3125 \end{cases}$$

Шаг $0.5 \to 1$:

$$\begin{cases} t_{1.5}=t_1+\frac{h}{2}f(x_1,t_1,s_1) \\ t_2=t_1+hf(x_{1.5},t_{1.5},s_{1.5}) \end{cases} \Leftrightarrow \begin{cases} t_{1.5}=1.3125+0.25\cdot(0.375\cdot 1.3125) \\ t_2=1.3125+0.5f(x_{1.5},t_{1.5},s_{1.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} t_{1.5}=1.435546875 \\ t_2=1.3125+0.5\cdot(0.08984375\cdot 1.435546875) \end{cases} \Leftrightarrow \begin{cases} t_{1.5}=1.435546875 \\ t_2=1.3769874573 \end{cases}$$

Получили значения $t^h = \{1, 1.3125, 1.3769874573\}$

Шаг 2: Решаем задачу Коши модифицированным методом Эйлера для нахождения y^h:

$$\begin{cases} y'(x)=s(x)y(x)+t(x) \\ y(1)=e=y_2 \end{cases}$$

Шаг 1 → 0.5 :

$$\begin{cases} y_{1.5}=y_2-\frac{h}{2}f(x_2,y_2,s_2,t_2) \\ y_1=y_2-hf(x_{1.5},y_{1.5},s_{1.5},t_{1.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} y_{1.5}=e-0.25\cdot(0.1290359497\cdot e+1.3769874573) \\ y_1=e-0.5f(x_{1.5},y_{1.5},s_{1.5},t_{1.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} y_{1.5}=2.2863459448 \\ y_1=e-0.5\cdot(-0.08984375\cdot 2.2863459448+1.435546875) \end{cases} \Leftrightarrow$$

$$\begin{cases} y_{1.5}=2.2863459448 \\ y_1=2.1032153377 \end{cases}$$

Шаг 0.5 → 0 :

$$\begin{cases} y_{0.5}=y_1-\frac{h}{2}f(x_1,y_1,s_1,t_1) \\ y_0=y_1-hf(x_{0.5},y_{0.5},s_{0.5},t_{0.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} y_{0.5}=2.1032153377-0.25\cdot(-0.375\cdot 2.1032153377+1.3125) \\ y_0=2.1032153377-0.5f(x_{0.5},y_{0.5},s_{0.5},t_{0.5}) \end{cases} \Leftrightarrow$$

$$\begin{cases} y_{0.5}=1.9722667756 \\ y_0=1.9712820316 \end{cases}$$

Получили сеточную функцию $y^h = \{y_0, y_1, y_2\} = \{1.9712820316, 2.1032153377, e\}$

Ошибки:

В точке 0: $|y_0 - y(x_0)| = |1.9712820316 - 4.1945280495| = 2.2232460179$

В точке 0.5: $|y_1 - y(x_1)| = |2.1032153377 - 3.06520517052| = 0.9619898328$

Подготовка контрольных тестов

1. Исследование зависимости погрешности от h.
   После получения результата по методу факторизации, считаем погрешность. На следующем шаге уменьшаем h и считаем погрешность по норме $\|y(x^h) - y^h\|$. h меняется до порядка $1e-4$.

2. Исследовать влияние ошибок в исходных данных (ГУ) на решение.

   Задаем δ равное 10^{-1} и далее уменьшаем его до порядка 10^{-11} и считаем получившуюся глобальную погрешность.

Модульная структура программы

- `double exactSolution(double x);`
  Описание: значение интегральной кривой в точке x
  - Входные данные:
    - Значение x, в котором считаем значение функции
  - Выходные данные:
    - Значение интегральной кривой $y = e^x$

- `std::vector<double> buildExactSolution(double a, double b, double h, std::function<double(double)> func);`
  Описание: сетка с шагом h на отрезке $[a, b]$ точной интегральной кривой
  - Входные данные:
    - a – левая граница отрезка
    - b – правая граница отрезка
    - h – шаг
    - func – указатель на функцию, для которой строим сетку
  - Выходные данные:
    - массив значений интегральной кривой

- `struct DifferentialEquation`
  Описание: структура, которая хранит указатели функций $p(x), q(x), r(x), f(x)$

- `struct BoundaryConditions`
  Описание: структура хранит значения ГУ – $\alpha_0, \alpha_1, \beta_0, \beta_1, A, B$

- `double methodEuler(double x_k, double s_k, double h, std::function<double(double, double)> func);`
  Описание: метод Эйлера для функции $s(x)$
  - Входные данные:
    - x_k – x на k-ом шаге
    - s_k – значение s на k-ом шаге
    - h – шаг
    - func – указатель на функцию $s^2 - \frac{q}{p}s - \frac{r}{p}$
  - Выходные данные:
    - значение $s_{k+0.5}$

- `double modifiedEulerMethod(double x_k, double s_k, double h, std::function<double(double, double)> func);`
  Описание: модифицированный метод Эйлера для функции $s(x)$
  - Входные данные:
    - x_k – x на k-ом шаге
    - s_k – значение s на k-ом шаге
    - h – шаг

  - func – указатель на функцию $s^2 - \frac{q}{p}s - \frac{r}{p}$
- Выходные данные:
  - значение s_{k+1}

- `double methodEuler(double x_k, double t_k, double s_k, double h, std::function<double(double, double, double)> func);`
  Описание: метод Эйлера для функции $t(x)$
  - Входные данные:
    - x_k – x на k-ом шаге
    - t_k – значение t на k-ом шаге
    - s_k – значение s на k-ом шаге
    - h – шаг
    - func – указатель на функцию $\frac{f}{p} - st - \frac{qt}{p}$
  - Выходные данные:
    - значение $t_{k+0.5}$

- `double modifiedEulerMethod(double x_k, double t_k, double s_k, double s_half, double h, std::function<double(double, double, double)> func);`
  Описание: модифицированный метод Эйлера для функции $t(x)$
  - Входные данные:
    - x_k – x на k-ом шаге
    - t_k – значение t на k-ом шаге
    - s_k – значение s на k-ом шаге
    - s_{half} – значение $s_{k+0.5}$
    - h – шаг
    - func – указатель на функцию $\frac{f}{p} - st - \frac{qt}{p}$
  - Выходные данные:
    - Значение t_k

- `double methodEulerBackwards(double x_k, double y_k, double s_k, double t_k, double h, std::function<double(double, double, double)> func);`
  Описание: метод Эйлера для функции $y(x)$ в обратную сторону
  - Входные данные:
    - x_k – x на k-ом шаге
    - y_k – значение y на k-ом шаге
    - s_k – значение s на k-ом шаге
    - t_k – значение t_k
    - h – шаг
    - func – указатель на функцию $sy + t$
  - Выходные данные:
    - значение $y_{k+0.5}$

- `double modifiedEulerMethodBackwards(double x_k, double y_k, double s_k, double s_half, double t_k, double t_half, double h, std::function<double(double, double, double)> func);`

  Описание: модифицированный метод Эйлера для функции $y(x)$ в обратную сторону

  ▸ Входные данные:
  - x_k – x на k-ом шаге
  - y_k – значение y на k-ом шаге
  - s_k – значение s на k-ом шаге
  - s_{half} – значение $s_{k+0.5}$
  - t_k – значение t_k
  - t_{half} – значение $t_{k+0.5}$
  - h – шаг
  - func – указатель на функцию $sy + t$

  ▸ Выходные данные:
  - значение y_{k+1}

- `std::vector<double> factorizationMethod(double a, double b, double h, const DifferentialEquation &equations, const BoundaryConditions &conditions, const double delta = 0.0);`

  Описание: ищет интегральную функцию методом факторизации

  ▸ Входные данные:
  - a – левая граница
  - b – правая граница
  - h – шаг
  - equations – структура с функциями $p(x), q(x), r(x), f(x)$
  - conditions – структура с ГУ
  - delta – параметр по умолчанию, отклонение вводящееся в начальное условие $y(x)$

  ▸ Выходные данные:
  - сетка значений интегральной кривой

- `double norm(const std::vector<double> exact, const std::vector<double> yMethod);`

  Описание: считает ошибку по норме $\|e\|_\infty = \max|y_i - y(x_i)|$

  ▸ Входные данные:
  - exact – массив точных значений в точках x_i
  - yMethod – массив численных значений в точках x_i

- Выходные данные:
  - полученная ошибка метода

- `double stepError(double y_method, double y_actual);`
  Описание: ошибка на шаге метода
  - Входные данные:
    - y_method – численное значение
    - y_actual – точное значение
  - Выходные данные:
    - полученная ошибка на шаге

- ```
  void RunExperiments(const std::string path,
  double a,
  double b,
  const DifferentialEquation &equations,
  const BoundaryConditions &conditions);
  ```
  Описание: запускает эксперименты для краевой задачи и записывает все полученные данные в .txt файлы
  - Входные данные:
    - path – путь куда будут сохраняться данные
    - a – левая граница
    - b – правая граница
    - equations – структура, которая хранит указатели на функции $p(x), q(x), r(x), f(x)$
    - conditions – структура, которая хранит граничные условия
  - Выходные данные:
    - .txt файлы

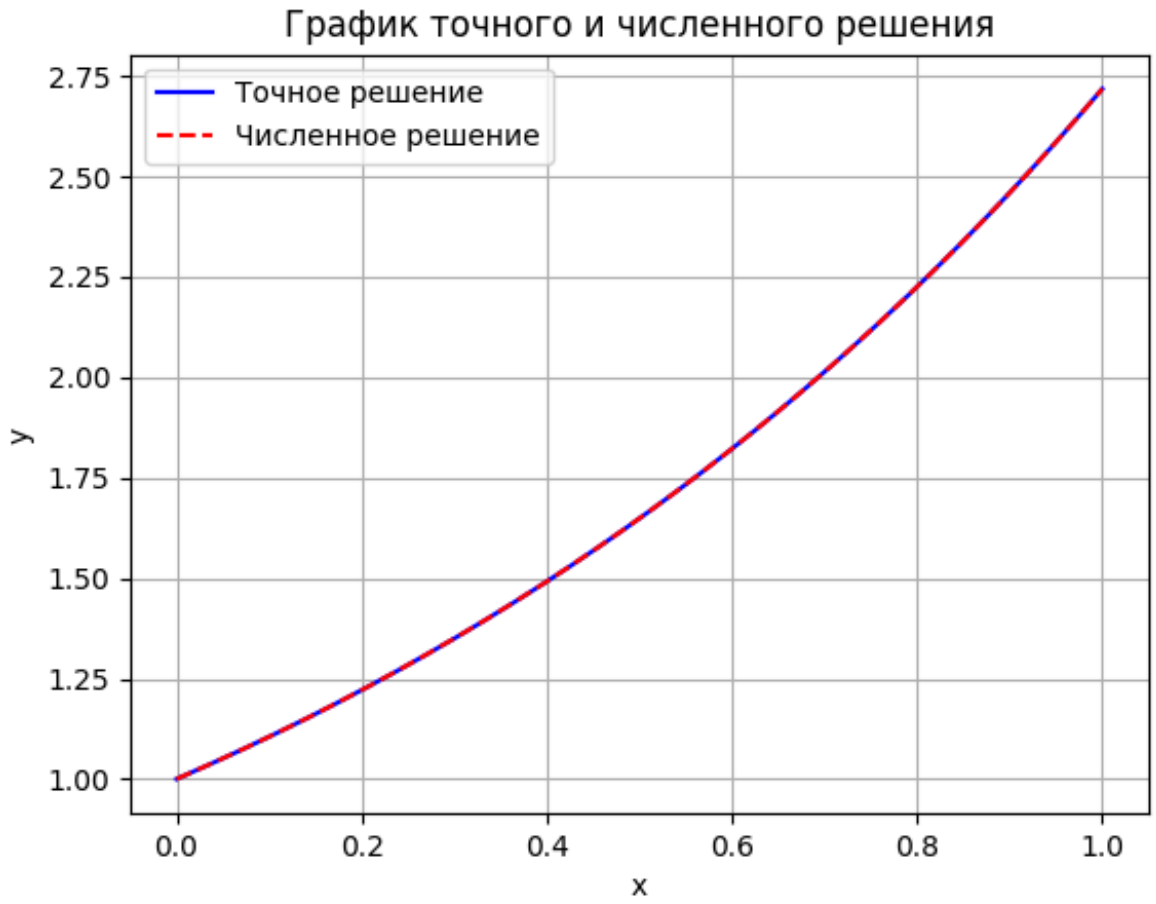
Численный анализ

Исследования, которые были проведены:

1. Зависимость ошибки от h
2. Влияние возмущения в исходных данных на ошибку
  ```

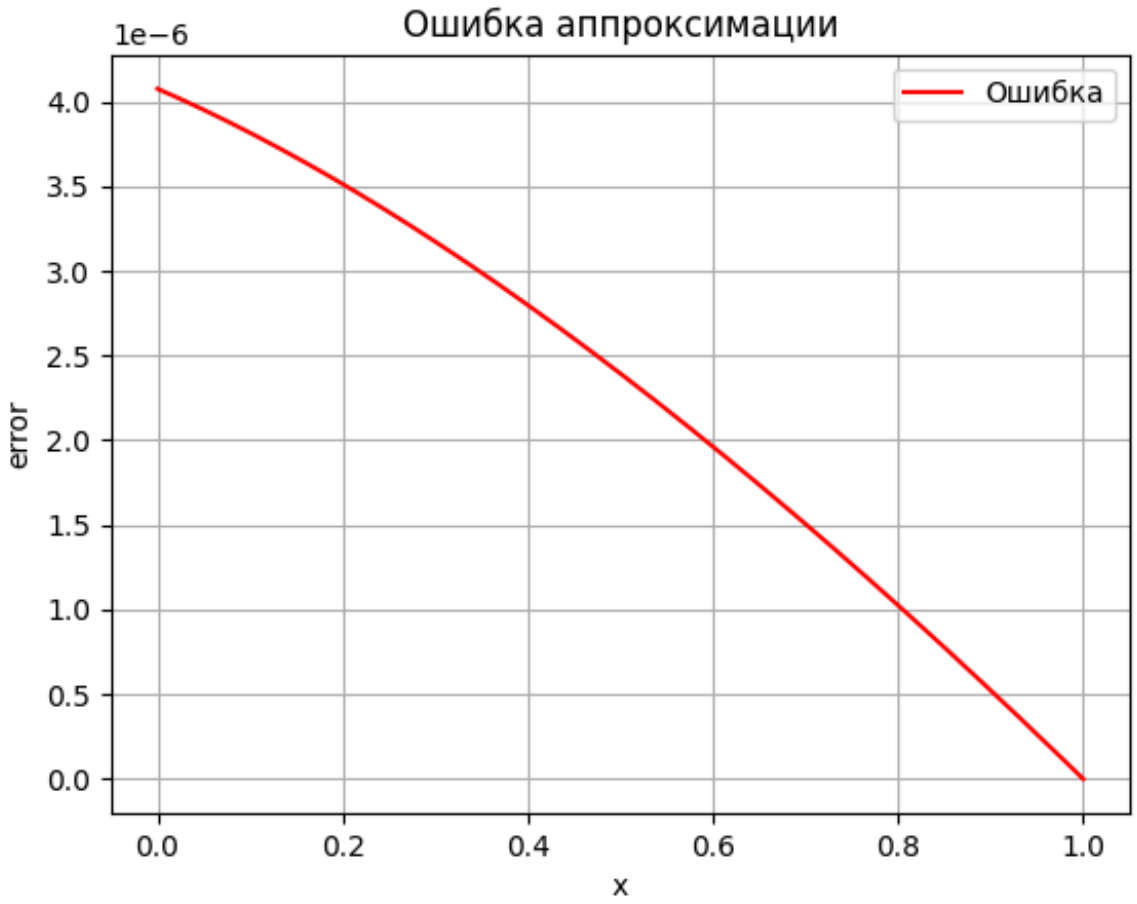
Графики

1. Численное решение методом факторизации ($h = 0.001$)

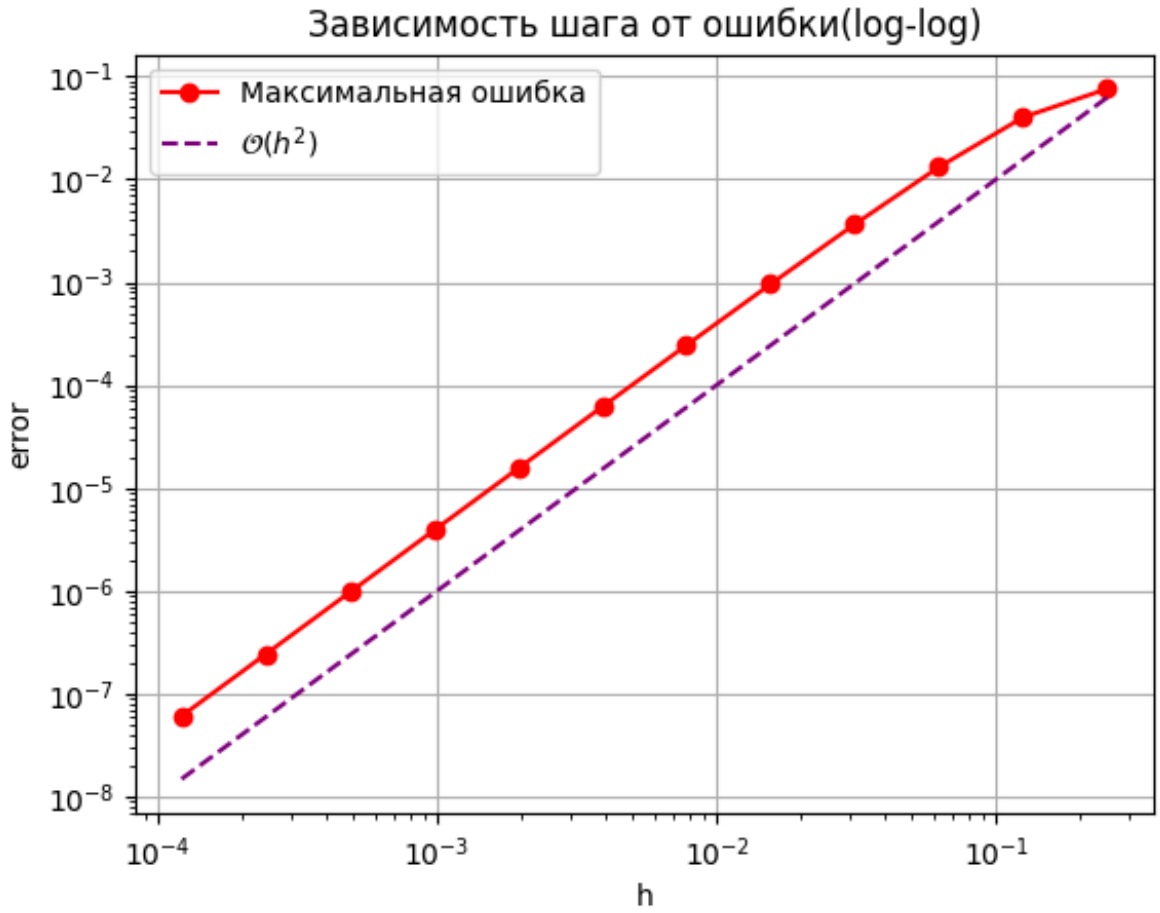


Из графика видно, что численное решение хорошо аппроксимирует точное решение.

2. Ошибка метода факторизации

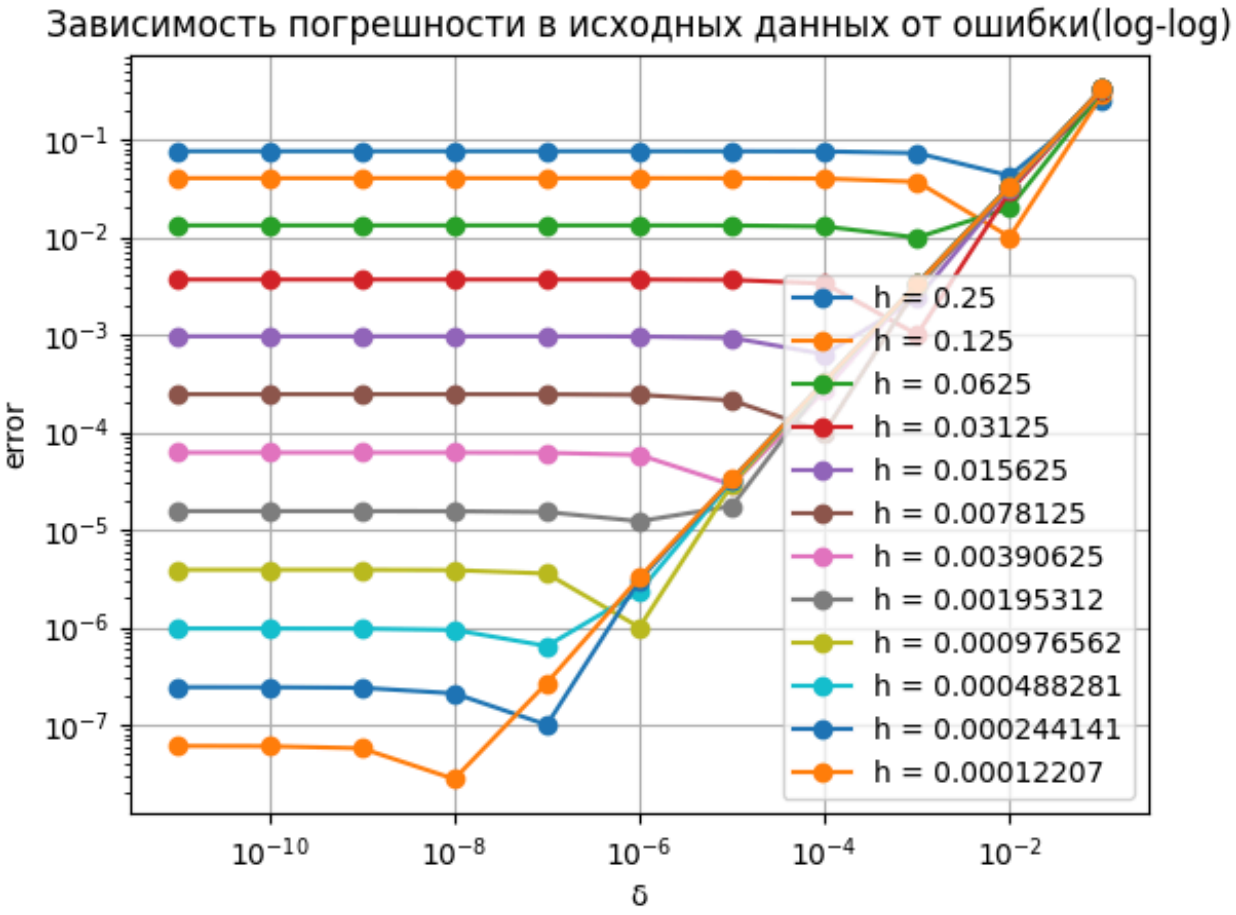


3. Зависимость ошибки от шага сетки



Из графика видно, что ошибка соответствует теории для модифицированного метода Эйлера

4. Устойчивость метода к возмущениям в исходных данных



Из графика видно, что метод факторизации является устойчивым

Вывод

Метод факторизации успешно применен для решения краевой задачи ОДУ второго порядка. Численные эксперименты подтвердили теоретическую оценку погрешности метода – погрешность постепенно закономерно уменьшается с уменьшением шага сетки.